# Développement d'applications web avancées

Informatique — Licence 3 (L3)

Cours INF-L3-03

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

**À propos de ce cours**
Concevoir des applications web modernes et découplées : architecture client/API/données, API REST soignée, authentification par jetons JWT et délégation OAuth, frameworks côté serveur et côté client, cache et performances, jusqu'au projet complet.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

# Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Architecture d'une application web moderne
• Leçon 2 — API REST : conception, ressources et codes HTTP
• Leçon 3 — Authentification par jetons JWT et initiation à OAuth
• Leçon 4 — Frameworks côté serveur : structure et conventions
• Leçon 5 — Frameworks côté client : composants et gestion d'état
• Leçon 6 — Cache, performances et montée en charge
• Leçon 7 — Projet : application complète communiquant avec une API
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

# Introduction à la matière

En Licence 2, tu as construit des pages web dynamiques en PHP, interrogé une base de données en SQL et assemblé une première application complète. Ce cours passe au niveau supérieur : celui des applications que les professionnels construisent aujourd'hui, où l'interface que voit l'utilisateur et le serveur qui détient les données sont deux programmes distincts qui se parlent par une API. C'est ce découplage qui permet à un même service d'alimenter un site web, une application mobile et parfois les systèmes d'autres entreprises.

Nous suivrons un fil rouge unique : BiblioIAM, l'API de la bibliothèque de l'institut, qui gère des livres, des membres et des emprunts. Chaque leçon ajoute une couche à cette application réelle dans l'esprit : d'abord son architecture, puis la conception de ses adresses et de ses réponses, puis la protection des comptes par jetons signés, puis l'organisation du code avec des frameworks des deux côtés, et enfin les réglages qui la rendent rapide quand le nombre d'utilisateurs grandit.

À la fin du cours, tu sauras lire un échange HTTP complet sans rien deviner, choisir le bon code de statut, expliquer pourquoi un jeton JWT n'a pas besoin d'être gardé secret mais doit être vérifié, et présenter un projet d'application complète communiquant avec une API. Ce sont exactement les compétences demandées dans les offres de stage et les entretiens de développeur web junior : ce cours est donc aussi une préparation directe à ton insertion professionnelle.

# Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Décrire l'architecture d'une application web moderne (clients, API, données) et suivre le trajet complet d'une requête.
- Concevoir une API REST : ressources nommées au pluriel, méthodes HTTP et codes de statut adaptés à chaque situation.
- Expliquer la structure d'un jeton JWT, le rôle de sa signature et sa vérification côté serveur, et situer OAuth 2.0.
- Distinguer authentification et autorisation et appliquer correctement les codes 401 et 403.
- Structurer un projet avec un framework côté serveur : routage, contrôleurs, modèles, intergiciels (middlewares).
- Découper une interface en composants réutilisables et organiser la gestion d'état côté client.
- Améliorer les performances d'une API (cache HTTP, pagination, requêtes) et mener un mini-projet complet de bout en bout.

# Leçon 1 — Architecture d'une application web moderne

Les premiers sites dynamiques mélangeaient tout dans un seul programme : le serveur générait les pages HTML, exécutait les règles métier et interrogeait la base de données. Les applications modernes séparent ces responsabilités en blocs qui communiquent par le réseau. D'un côté, un ou plusieurs clients : une page web exécutée dans le navigateur, une application mobile, parfois un logiciel de bureau. De l'autre, un serveur d'API qui expose les données et les règles métier. Entre les deux, uniquement des requêtes HTTP et des réponses, le plus souvent au format JSON.

Côté serveur, on distingue encore plusieurs couches. La couche d'accès aux données dialogue avec la base (livres, membres, emprunts) et ne connaît rien aux pages. La couche métier applique les règles : un livre déjà emprunté ne peut pas l'être une seconde fois, un emprunt dure quatorze jours, un membre en retard est bloqué. La couche API reçoit les requêtes, vérifie qui parle et ce qu'il demande, appelle le métier et formate la réponse. Cette séparation permet de tester chaque couche isolément et de faire évoluer l'une sans casser les autres.

Ce découplage a trois grandes conséquences pratiques. D'abord, les clients se multiplient sans réécrire le serveur : le site web des étudiants et l'application mobile des bibliothécaires utilisent la même API BiblioIAM. Ensuite, chaque bloc monte en charge séparément : si les consultations du catalogue explosent, on renforce l'API sans toucher à l'interface. Enfin, les équipes se répartissent le travail : les développeurs front-end avancent sur l'interface pendant que les développeurs back-end construisent l'API, en se mettant d'accord sur un contrat : la liste des adresses et des formats d'échange. Tout le reste du cours détaille ce contrat et sa mise en œuvre.

**L'essentiel**
Une application moderne = des clients (web, mobile) qui parlent en HTTP/JSON à une API, elle-même organisée en couches (API, métier, données). Le contrat entre clients et serveur, c'est la liste des routes et des formats : tout le monde s'y tient.

## Exemple — L'architecture de BiblioIAM et le trajet d'un emprunt

```
L'application BiblioIAM, en quatre blocs

[ Navigateur ] [ Application mobile ]
(interface web) (interface Android)
\ /
\ HTTPS /
v v
+--------------------------------------+
| API BiblioIAM (serveur) |
| - routes et règles métier |
| - vérification des jetons |
+--------------------------------------+
|
v
+--------------------------------------+
| Base de données (livres, membres, |
| emprunts) - accès réservé à l'API |
+--------------------------------------+

Trajet d'une demande d'emprunt :
1. Le membre clique "Emprunter" dans l'interface.
2. Le client envoie POST /v1/emprunts + son jeton.
3. L'API vérifie le jeton, puis la disponibilité.
4. L'API enregistre l'emprunt dans la base.
5. Réponse 201 : le client affiche la date de retour.
```

## À retenir

- Client et serveur sont des programmes distincts : ils ne partagent que le contrat de l'API (routes et formats JSON).
- Trois couches côté serveur : API (requêtes/réponses), métier (règles), données (base) — chacune testable séparément.
- La base de données n'est jamais exposée directement aux clients : tout passe par l'API, qui contrôle les accès.

## Exercice

Une école veut moderniser la gestion de sa cantine : les élèves consulteront le menu et réserveront leur repas depuis leur téléphone, la cantinière gérera les plats depuis un ordinateur, et la direction veut des statistiques de fréquentation. Découpe ce projet selon l'architecture de la leçon : quels clients, quelle API, quelles couches côté serveur, et quelles sont les trois ressources principales manipulées ? Écris le trajet complet d'une réservation de repas, en cinq étapes numérotées comme dans l'exemple.

**✔ Corrigé**
Clients : une application mobile pour les élèves (consulter le menu, réserver), une interface web pour la cantinière (gérer plats et réservations) — les statistiques de la direction peuvent être un écran de l'interface web, alimenté par la même API. Serveur : une API unique « CantineIAM » en trois couches — couche API (routes et vérification des jetons d'élèves), couche métier (règles : une réservation par élève et par jour, plat indisponible quand le stock est épuisé, annulation possible jusqu'à 9 h), couche données (base des plats, élèves, réservations). Ressources principales : les plats, les élèves et les réservations. Trajet d'une réservation : 1) l'élève choisit un plat dans l'application ; 2) le client envoie POST /v1/reservations avec son jeton et l'identifiant du plat ; 3) l'API vérifie le jeton, puis la règle métier (pas déjà de réservation ce jour, stock suffisant) ; 4) l'API enregistre la réservation et décrémente le stock ; 5) réponse 201 : l'application affiche le numéro de réservation. Point clé du corrigé : la règle « une seule réservation par jour » vit dans la couche métier, pas dans l'interface — sinon un élève habile pourrait la contourner en appelant l'API directement.

# Leçon 2 — API REST : conception, ressources et codes HTTP

REST est un style d'architecture pour les API web, formalisé par Roy Fielding en 2000. Son idée centrale : le serveur expose des ressources — les objets du métier — désignées par des adresses (URL) stables, et le client agit sur ces ressources avec les méthodes standard du protocole HTTP. Une ressource se nomme par un nom, au pluriel, sans verbe : /livres, /membres, /emprunts. L'action est portée par la méthode : GET pour lire, POST pour créer, PUT ou PATCH pour modifier, DELETE pour supprimer. Écrire /recupererLivres ou /livre/supprimer/7, c'est mélanger les rôles et perdre tous les avantages du style.

Deux propriétés structurent une API REST bien conçue. La première est l'absence d'état (stateless) : le serveur ne conserve aucune session entre les requêtes ; chaque requête contient tout ce qu'il faut pour être comprise, y compris le jeton qui identifie l'appelant (leçon 3). C'est ce qui permet de répartir les requêtes sur plusieurs serveurs sans se soucier de « qui a parlé à qui avant ». La seconde est l'uniformité des réponses : un même format JSON partout, une pagination explicite pour les listes (page, par_page) plutôt que des milliers d'objets d'un bloc, et des filtres passés en paramètres de requête, comme /livres?disponible=true.

Le troisième pilier est le vocabulaire des codes de statut HTTP, défini par la RFC 9110. Ils racontent le résultat sans ambiguïté : 200 OK pour une lecture ou une action réussie, 201 Created pour une création (avec l'adresse de la nouvelle ressource), 204 No Content pour une suppression réussie sans corps de réponse. Côté erreurs du client : 400 Bad Request si la demande est mal formée, 401 Unauthorized si l'identité n'est pas prouvée, 403 Forbidden si elle est prouvée mais insuffisante, 404 Not Found si la ressource n'existe pas, 409 Conflict si la demande contredit l'état actuel (emprunter un livre déjà emprunté). Le 500 Internal Server Error, lui, signale une panne du serveur — jamais une erreur de l'utilisateur. Un corps d'erreur en JSON, avec un message clair, complète le code : le client doit pouvoir afficher quelque chose d'utile à l'humain.

**L'essentiel**
REST = des ressources nommées par des noms au pluriel (/livres, /emprunts), des méthodes HTTP qui portent l'action, aucune session côté serveur, et des codes de statut qui disent exactement ce qui s'est passé (200, 201, 400, 401, 403, 404, 409, 500).

## Exemple — Échanges complets avec l'API BiblioIAM : succès et erreurs

```
--- Consulter le catalogue (public) ---
GET /v1/livres?disponible=true HTTP/1.1
Host: api.biblio-iam.ml
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

[ { "id": 7, "titre": "Réseaux et protocoles",
"auteur": "F. Martin", "disponible": true } ]

--- Emprunter ce livre (jeton obligatoire) ---
POST /v1/emprunts HTTP/1.1
Host: api.biblio-iam.ml
Authorization: Bearer eyJhbGciOi... (jeton)
Content-Type: application/json

{ "livre_id": 7 }

HTTP/1.1 201 Created
Location: /v1/emprunts/58
Content-Type: application/json

{ "id": 58, "livre_id": 7, "membre_id": 42,
"date_retour": "2026-10-29" }

--- Le même livre demandé deux fois ---
POST /v1/emprunts { "livre_id": 7 }
HTTP/1.1 409 Conflict
{ "erreur": "Livre déjà emprunté" }

--- Trois erreurs à ne pas confondre ---
GET /v1/livres/999 -> 404 Not Found
POST /v1/emprunts (sans jeton) -> 401 Unauthorized
POST /v1/emprunts { "livre": "7" }
-> 400 Bad Request (champ "livre_id" entier attendu)
```

## À retenir

• L'URL nomme la ressource (nom au pluriel), la méthode HTTP porte l'action : jamais de verbe dans l'URL.

• Sans état : chaque requête est autonome et contient son jeton ; le serveur ne se souvient pas des conversations.

• 201 pour une création, 409 pour un conflit d'état, 401 identité non prouvée, 404 ressource absente : chaque code a un sens précis.

### Exercice

Un collègue a conçu les adresses suivantes pour une boutique en ligne : GET /recupererProduits, POST /produit/ajouter, GET /commande.php?id=12, POST /supprimerProduit/5, GET /client/commandes/listeComplete. Réécris chacune de ces cinq adresses en style REST correct (méthode + URL), en supposant les ressources produits, clients et commandes. Pour l'une d'elles, précise en plus le code de statut d'une réponse réussie.

**✔ Corrigé**

1) GET /recupererProduits devient GET /produits : le verbe disparaît, la ressource est au pluriel, réponse 200 avec la liste (paginée). 2) POST /produit/ajouter devient POST /produits : la création est portée par POST sur la collection, réponse 201 Created avec l'adresse du nouveau produit dans l'en-tête Location. 3) GET /commande.php?id=12 devient GET /commandes/12 : l'identifiant fait partie de l'adresse de la ressource, pas d'un paramètre, et l'extension de fichier n'a rien à faire dans une URL d'API. 4) POST /supprimerProduit/5 devient DELETE /produits/5 : la suppression a sa propre méthode HTTP, réponse 204 No Content. 5) GET /client/commandes/listeComplete devient GET /clients/{id}/commandes, par exemple GET /clients/9/commandes : les commandes d'un client sont une sous-collection de la ressource client, et « listeComplete » est un verbe déguisé inutile. Retiens la méthode de correction : d'abord repérer la ressource (un nom), puis choisir la méthode selon l'action, puis vérifier le code de statut.

## Leçon 3 — Authentification par jetons JWT et initiation à OAuth

Une API sans état ne peut pas utiliser les sessions classiques : il n'y a pas de mémoire partagée entre les requêtes. La solution la plus répandue est le jeton JWT (JSON Web Token, norme RFC 7519). Le scénario est le suivant : le membre envoie une fois son e-mail et son mot de passe à la route de connexion ; le serveur vérifie le mot de passe en le comparant à l'empreinte hachée stockée en base (bcrypt ou argon2 — jamais le mot de passe en clair, ni stocké ni comparé) ; si tout est bon, le serveur fabrique un jeton signé et le renvoie. Le client joint ensuite ce jeton à chaque requête dans l'en-tête Authorization: Bearer <jeton>.

Un JWT comporte trois parties séparées par des points : en-tête . charge utile . signature. L'en-tête annonce l'algorithme (par exemple HS256). La charge utile (payload) contient des affirmations (claims) : sub, l'identifiant du sujet ; iat, la date d'émission ; exp, la date d'expiration ; plus des informations utiles comme le nom ou le rôle. En-tête et charge sont simplement encodés en Base64URL : n'importe qui peut les lire — un JWT n'est pas chiffré, il est signé. La signature est calculée par le serveur avec une clé secrète sur les deux premières parties : si quelqu'un modifie la charge (par exemple pour se déclarer administrateur), la signature ne correspond plus et le serveur rejette le jeton avec un 401. La clé secrète ne quitte jamais le serveur, et aucune donnée confidentielle ne doit figurer dans la charge utile, précisément parce

qu'elle est lisible par tous.

Deux distinctions évitent la plupart des confusions. Authentification et autorisation ne sont pas synonyms : l'authentification répond à « qui es-tu ? » (jeton valide), l'autorisation à « as-tu le droit ? » (rôle suffisant) — d'où les codes 401 (identité non prouvée) et 403 (identité prouvée, droit insuffisant). Ensuite, les jetons ont des limites : difficiles à révoquer avant expiration (on les fait donc courts, une heure par exemple, avec éventuellement un jeton de rafraîchissement), et volables si la connexion n'est pas protégée — HTTPS est obligatoire. Enfin, OAuth 2.0 répond à un autre besoin : déléguer l'authentification. Au lieu de créer un compte de plus, l'utilisateur se connecte via un fournisseur d'identité (Google, Microsoft, l'université) qui, après son accord, délivre un jeton d'accès à l'application : c'est le bouton « Se connecter avec Google », et OpenID Connect ajoute à OAuth la preuve standardisée de l'identité.

**L'essentiel**
Un JWT = en-tête . charge utile . signature, encodé en Base64URL : lisible par tous, infalsifiable sans la clé secrète du serveur, qui vérifie signature et expiration à chaque requête. Le mot de passe, lui, n'est stocké que sous forme hachée (bcrypt/argon2).

## Exemple — Un vrai jeton JWT, décodé champ par champ

```
Jeton émis par le serveur BiblioIAM pour Awa,
membre n° 42 (clé de démonstration, algorithme HS256)

En-tête (Base64URL) :
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
-> { "alg": "HS256", "typ": "JWT" }

Charge utile (Base64URL, découpée pour l'affichage) :
eyJzdWIiOiI0MiIsIm5vbSI6IkF3YSBEaWFs
bG8iLCJyb2xlIjoibWVtYnJlIiwiaWF0Ijox
NzYwMDAwMDAwLCJleHAiOjE3NjAwMDM2MDB9
-> { "sub": "42", "nom": "Awa Diallo",
"role": "membre",
"iat": 1760000000, "exp": 1760003600 }

Signature (Base64URL) :
jW3zTff7ViemP9N58xAhVBmcb8Sn5J9lLkSap9_v49w
= HMAC-SHA256(en-tête + "." + charge,
clé secrète du serveur)

Lecture des durées : exp - iat = 3600 s = 1 heure.
Test de falsification : si l'on remplace "membre"
par "admin" dans la charge sans connaître la clé,
la signature recalculée est différente -> le
serveur répond 401. La charge est LISIBLE par
tous mais MODIFIABLE par personne sans la clé.
```

## À retenir

- Base64URL n'est pas un chiffrement : la charge d'un JWT se lit comme un texte clair — jamais de secret dedans.

• La signature protège l'intégrité : toute modification de la charge la rend invalide ; le serveur vérifie signature ET expiration.

• 401 = qui es-tu ? (authentification) ; 403 = tu n'as pas le droit (autorisation). OAuth sert à déléguer l'authentification à un fournisseur d'identité.

### Exercice

On intercepte le jeton d'un étudiant et on lit sa charge utile : { "sub": "17", "nom": "Moussa Keita", "role": "membre", "iat": 1760000000, "exp": 1760003600 }. 1) Sans la clé secrète, peut-on utiliser ce jeton tel quel pour appeler l'API ? 2) Peut-on le transformer pour devenir bibliothécaire (role: "bibliothecaire") ? 3) Le serveur doit-il accepter ce jeton trois jours après son émission ? Justifie chaque réponse en citant le mécanisme concerné.

**✔ Corrigé**

1) Oui, tel quel, le jeton est utilisable par quiconque le possède jusqu'à son expiration : c'est le principe du « Bearer » (porteur) — posséder le jeton suffit. D'où deux protections indispensables : HTTPS pour empêcher l'interception, et une durée de vie courte. 2) Non : modifier la charge utile change le résultat du calcul de signature ; le serveur, qui recalcule la signature avec sa clé secrète, constate l'écart et rejette la requête par un 401. Devenir bibliothécaire par ce moyen exigerait de connaître la clé secrète du serveur. 3) Non : exp − iat = 3 600 secondes, soit une heure de validité. Trois jours plus tard, le jeton est expiré ; même avec une signature correcte, le serveur le refuse (401) et l'étudiant doit se reconnecter pour en obtenir un nouveau. Remarque : la lisibilité de la charge n'est pas une faille — elle est voulue par la norme ; la sécurité repose entièrement sur la signature et la courte durée de vie.

## Leçon 4 — Frameworks côté serveur : structure et conventions

Un framework côté serveur fournit le squelette que toutes les applications web partagent : recevoir une requête HTTP, la diriger vers le bon traitement (le routage), lire les paramètres et le corps, appliquer des traitements transverses (journalisation, vérification du jeton), produire une réponse. Sans framework, chaque projet réinvente ces mécanismes avec ses propres erreurs. Les frameworks connus — Laravel pour PHP, Django et Flask pour Python, Express pour Node.js, Spring pour Java — partagent la même philosophie : des conventions fortes (« convention plutôt que configuration ») qui disent où ranger chaque chose, pour que tout développeur retrouve ses marques d'un projet à l'autre.

L'organisation la plus répandue découpe le serveur en rôles nets, dans l'esprit du modèle MVC adapté aux API. Le routeur associe chaque couple (méthode, URL) à un traitement : POST /v1/emprunts va vers la fonction creer_emprunt. Le contrôleur, mince, lit la demande, appelle le métier et choisit la réponse (code et JSON). Le modèle représente les données et leurs règles d'accès à la base. Autour, les intergiciels (middlewares) forment une chaîne que chaque requête traverse : enregistrer la demande, vérifier le jeton, puis seulement atteindre le contrôleur. Enfin, la validation des entrées (types, champs obligatoires) se fait à l'entrée du contrôleur : ce sont ces

couches qui transforment proprement une demande incorrecte en 400 plutôt qu'en panne 500.

Choisir un framework ne se fait pas au hasard : on regarde le langage déjà maîtrisé par l'équipe, la qualité de la documentation, la taille de la communauté (donc la facilité à trouver réponses et bibliothèques), et l'adéquation au projet — un micro-framework léger pour une petite API, un framework complet quand il faut authentification, administration et files de tâches. Mais un framework ne conçoit pas l'application à ta place : les ressources, les règles métier et les codes de statut restent tes décisions. Il donne des rails, pas la destination — et c'est exactement ce qui rend le code d'un projet à l'autre si facile à reprendre.

**L'essentiel**
Un framework serveur fournit routage, gestion des requêtes/réponses, intergiciels et conventions de rangement (contrôleurs minces, modèles pour les données). Il fait gagner du temps et de la fiabilité, mais la conception de l'API reste le travail du développeur.

## Exemple — Deux routes de BiblioIAM dans un style de framework Python (extrait)

```
# Extrait du serveur BiblioIAM (style Flask)
from flask import Flask, request, jsonify
from werkzeug.security import check_password_hash

app = Flask(__name__)

@app.route("/v1/connexion", methods=["POST"])
def connexion():
donnees = request.get_json()
membre = trouver_membre(donnees.get("email"))
if membre is None:
return jsonify(erreur="Identifiants invalides"), 401
empreinte = membre["empreinte"] # hachée en base
if not check_password_hash(empreinte,
donnees.get("mot_de_passe", "")):
return jsonify(erreur="Identifiants invalides"), 401
jeton = creer_jeton(membre) # signé, valable 1 h
return jsonify(jeton=jeton), 200

@app.route("/v1/emprunts", methods=["POST"])
@jeton_requis # intergiciel : 401 si absent
def creer_emprunt():
livre_id = request.get_json().get("livre_id")
if not isinstance(livre_id, int):
return jsonify(erreur="livre_id entier requis"), 400
if not livre_disponible(livre_id):
return jsonify(erreur="Livre déjà emprunté"), 409
emprunt = enregistrer_emprunt(request.membre_id,
livre_id)
return jsonify(emprunt), 201
```

## À retenir

• Le routeur envoie chaque (méthode, URL) vers un contrôleur ; les intergiciels traitent ce qui est commun (jeton, journaux) avant le contrôleur.

• Le contrôleur reste mince : il valide l'entrée, appelle le métier, choisit le code de statut et le JSON de réponse.

• Validation à l'entrée = erreurs 400 propres ; une demande incorrecte ne doit jamais devenir une panne 500.

### Exercice

Dans un projet de framework, on te donne ces éléments en désordre : (a) le fichier qui associe POST /v1/livres à une fonction ; (b) la fonction qui vérifie la présence du jeton avant chaque route protégée ; (c) la fonction qui insère un livre dans la base ; (d) la fonction qui lit le corps JSON, vérifie que le titre est présent et répond 201. Pour chacun, dis son rôle exact (routeur, intergiciel, modèle, contrôleur) et justifie en une phrase. Puis explique pourquoi le contrôleur (d) ne doit pas contenir lui-même le code SQL d'insertion.

**✔ Corrigé**

(a) Routeur : c'est la table de correspondance entre adresses et traitements. (b) Intergiciel (middleware) : il s'exécute pour toutes les routes protégées, avant les contrôleurs, et peut interrompre la chaîne par un 401. (c) Modèle (couche données) : il est le seul à parler à la base ; le reste de l'application ignore le SQL. (d) Contrôleur : il orchestre la demande — validation, appel au modèle, choix du code 201 et du corps de réponse. Le contrôleur ne doit pas contenir le SQL car cela mélangerait trois responsabilités : si la structure de la table change, il faudrait corriger tous les contrôleurs au lieu d'un seul modèle ; le SQL dispersé devient impossible à tester isolément ; et les règles de validation se retrouveraient dupliquées. C'est la séparation des couches de la leçon 1, appliquée à l'intérieur du serveur : chaque rôle change pour une seule raison.

## Leçon 5 — Frameworks côté client : composants et gestion d'état

Côté navigateur, l'évolution est parallèle : au lieu de pages entières renvoyées par le serveur à chaque clic, les applications monopages (SPA, Single Page Application) chargent une seule page HTML, puis du JavaScript qui appelle l'API et met à jour uniquement la partie d'écran concernée. L'expérience devient fluide — plus de rechargement complet — mais le navigateur exécute désormais une vraie application, avec ses propres problèmes d'organisation. Les frameworks côté client — React, Vue.js, Angular sont les plus connus — apportent la réponse : découper l'interface en composants.

Un composant est un morceau d'interface autonome et réutilisable : il reçoit des données en entrée (ses propriétés), affiche quelque chose, et signale les actions de l'utilisateur par des événements. Une carte de livre, une barre de recherche, un formulaire de connexion sont des composants ; l'écran du catalogue est un assemblage de composants. Le même composant « carte de livre » sert dans le catalogue, dans les résultats de recherche et dans l'écran des emprunts, avec des

données différentes à chaque fois. Quand les données d'un composant changent, le framework recalcule son affichage automatiquement : le programmeur décrit « quoi afficher selon l'état », le framework s'occupe du « comment mettre l'écran à jour ».

Reste la question de l'état (state) : l'ensemble des données qui décrivent la situation de l'application à un instant donné. Certaines données sont locales à un composant — le texte en cours de frappe dans la barre de recherche n'intéresse qu'elle. D'autres sont globales : le membre connecté, son jeton, la liste de ses emprunts servent à de nombreux écrans ; on les place dans un magasin d'état partagé, alimenté par les réponses de l'API. Chaque appel à l'API traverse en outre trois moments que l'interface doit traiter honnêtement : chargement (afficher une attente), succès (afficher les données), erreur (afficher un message et proposer de réessayer). Une application qui ignore ces états semble « cassée » dès que le réseau ralentit.

**L'essentiel**
Les frameworks client découpent l'écran en composants réutilisables qui affichent des données et émettent des événements. L'état local vit dans le composant ; l'état partagé (utilisateur, jeton, emprunts) vit dans un magasin global alimenté par l'API, avec ses trois moments : chargement, succès, erreur.

## Exemple — Un composant, un magasin d'état, trois affichages (pseudocode)

```
// Composant d'interface (pseudocode, esprit
// commun aux frameworks React, Vue et Angular)
composant CarteLivre(livre, auClicEmprunter) :
afficher :
<carte>
<titre> {livre.titre} </titre>
<auteur> {livre.auteur} </auteur>
si livre.disponible :
<bouton onClick={auClicEmprunter}>
Emprunter
</bouton>
sinon :
<mention> Déjà emprunté </mention>
</carte>

// État global de l'application (partagé
// par tous les écrans)
etat = {
membreConnecte : null, // ou { id, nom, role }
jeton : null, // reçu à la connexion
mesEmprunts : [] // remplis par l'API
}

// Écran Catalogue : trois affichages selon
// le déroulement de l'appel à l'API
si chargement -> "Chargement des livres..."
sinon si erreur -> "Service indisponible,
réessayez dans un instant."
sinon -> une CarteLivre par livre reçu
```

### À retenir

• Un composant = des données en entrée, un affichage, des événements en sortie ; il se réutilise partout avec des données différentes.

• État local (un champ de saisie) dans le composant ; état global (membre connecté, jeton) dans un magasin partagé.

• Tout appel à l'API impose trois affichages : chargement, succès, erreur — les oublier donne des écrans figés ou vides.

### Exercice

Découpe l'écran « Mes emprunts » de BiblioIAM en composants : pour chacun, donne son nom, les données qu'il reçoit et les événements qu'il peut émettre. L'écran affiche l'identité du membre connecté, la liste de ses emprunts en cours (titre du livre, date de retour, bouton « Rendre »), et un message spécial quand la liste est vide. Précise ensuite quelles données relèvent de l'état global et lesquelles peuvent rester locales à un composant.

**✔ Corrigé**
Composants : 1) BandeauMembre — reçoit le membre connecté (nom, rôle), n'émet aucun événement (ou « déconnexion » si un bouton est prévu). 2) ListeEmprunts — reçoit le tableau des emprunts, émet rien elle-même mais contient les lignes. 3) LigneEmprunt — reçoit un emprunt (titre du livre, dates), émet l'événement « rendre » avec l'identifiant de l'emprunt quand on clique sur son bouton. 4) MessageListeVide — reçoit un booléen, affiche « Vous n'avez aucun emprunt en cours » quand la liste est vide. État global : le membre connecté et son jeton (utilisés par tous les écrans et par chaque appel à l'API), ainsi que la liste des emprunts si d'autres écrans la consultent (le catalogue s'en sert pour marquer les livres empruntés par le membre). État local : par exemple un éventuel indicateur « retour en cours… » sur la ligne cliquée, ou le texte d'un champ de filtre — ces données ne concernent que leur composant et disparaissent avec lui. Le principe de tri : si deux écrans ont besoin de la donnée, elle est globale ; sinon, elle reste au plus près de son composant.

## Leçon 6 — Cache, performances et montée en charge

Une application correcte mais lente perd ses utilisateurs : chaque seconde d'attente fait chuter l'usage. Optimiser commence par mesurer : où part le temps d'une requête ? Dans le réseau (surtout sur des connexions mobiles modestes), dans les allers-retours vers la base de données, dans des calculs répétés. Trois familles de réponses existent. Réduire le volume : paginer les listes (20 livres par page, pas 2 000), ne renvoyer que les champs utiles. Réduire les allers-retours : le piège classique est la requête « N+1 » — une requête pour lister 20 emprunts, puis une requête par emprunt pour trouver son livre, soit 21 requêtes là où une seule avec jointure suffit. Réduire le travail répété : c'est le rôle du cache.

Le cache conserve une copie d'un résultat pour éviter de le recalculer ou de le renvoyer. Le web possède un cache intégré au protocole HTTP : quand le serveur

répond avec les en-têtes Cache-Control (durée de validité) et ETag (une empreinte du contenu), le navigateur ou un proxy conserve la réponse. À la demande suivante, le client présente l'empreinte (If-None-Match) : si le contenu n'a pas changé, le serveur répond 304 Not Modified, sans corps — quelques octets au lieu du catalogue entier. Côté serveur, un cache en mémoire évite de recalculer les pages ou les listes les plus demandées. La difficulté du cache est son envers : l'invalidation. Une copie périmée qui survit trop longtemps montre un livre « disponible » alors qu'il vient d'être emprunté ; on choisit donc des durées courtes pour les données changeantes, et aucune mise en cache pour les données personnelles.

Quand le public grandit encore, on passe à la montée en charge horizontale : plusieurs exemplaires du serveur d'API derrière un répartiteur de charge, chacun pouvant traiter n'importe quelle requête. C'est là que le choix « sans état » de la leçon 2 paie : aucun serveur ne détient de session, le jeton voyage avec la requête, donc le répartiteur n'a pas à se souvenir de qui a parlé à quel serveur. On complète par un CDN (réseau de diffusion) qui rapproche les fichiers statiques des utilisateurs, par des index sur les colonnes recherchées de la base, et par la surveillance : mesurer en permanence les temps de réponse pour détecter la dégradation avant les utilisateurs. Retiens l'ordre de travail : mesurer, corriger les requêtes, paginer, cacher, puis seulement ajouter des serveurs.

**L'essentiel**
Performances : mesurer d'abord, puis paginer, tuer les requêtes N+1, et cacher ce qui change peu (en-têtes Cache-Control et ETag, réponse 304). La montée en charge horizontale n'est possible que parce que l'API est sans état : le jeton voyage avec chaque requête.

## Exemple — Le cache HTTP en action sur le catalogue BiblioIAM

```
--- Première demande du catalogue ---
GET /v1/livres HTTP/1.1
Host: api.biblio-iam.ml

HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: public, max-age=60
ETag: "livres-v42"
[ ... le catalogue complet ... ]

--- Une minute plus tard, même client ---
GET /v1/livres HTTP/1.1
Host: api.biblio-iam.ml
If-None-Match: "livres-v42"

HTTP/1.1 304 Not Modified
(aucun corps : le client garde sa copie)

Lecture : l'ETag est une empreinte du contenu.
Si le catalogue change, l'empreinte change et
le serveur renvoie à nouveau un 200 complet.

Autres réglages vus dans cette leçon :
- Pagination : GET /v1/livres?page=2&par_page=20
(jamais "tous les livres" d'un seul bloc).
- Index sur la colonne titre : recherche rapide.
- Une seule requête pour les emprunts et leurs
livres, au lieu d'une requête par emprunt (N+1).
```

## À retenir

• Ordre d'optimisation : mesurer, corriger les requêtes (N+1), paginer, mettre en cache, puis ajouter des serveurs.

• ETag + If-None-Match : si le contenu n'a pas changé, un 304 sans corps remplace le téléchargement complet.

• Le cache a une contrepartie : l'invalidation. Données personnelles et données changeantes = durées courtes ou pas de cache.

## Exercice

L'écran « Historique des emprunts » d'une bibliothèque affiche 300 emprunts et met 6 secondes à s'ouvrir. L'analyse montre : l'API renvoie les 300 emprunts en une réponse de 2 Mo ; pour chacun, le client fait ensuite un appel pour obtenir le nom du membre (300 appels) ; la réponse ne contient aucun en-tête de cache ; la recherche par nom de membre dans la base n'utilise aucun index. Propose un plan d'optimisation en quatre actions ordonnées, en reliant chacune au problème mesuré qu'elle corrige, et précise le mécanisme de la leçon qu'elle emploie.

**✔ Corrigé**
Action 1 — supprimer les appels en cascade (requêtes N+1) : l'API doit joindre le nom du membre à chaque emprunt dans la réponse initiale (une requête avec jointure côté serveur, ou un champ « membre_nom » ajouté à la représentation). On passe de 301 requêtes à 1 : c'est le gain le plus massif, et il ne coûte aucune infrastructure. Action 2 — paginer : renvoyer 25 emprunts par page avec les paramètres page et par_page, plus le total ; la réponse tombe de 2 Mo à environ 170 Ko et l'écran s'affiche dès la première page. Action 3 — indexer la colonne du nom (ou de l'identifiant de membre) dans la base : la jointure et les recherches cessent de parcourir toute la table à chaque fois. Action 4 — mettre en cache ce qui peut l'être : l'historique d'un membre change rarement en cours de journée ; un ETag sur la réponse permet des 304 pour les consultations répétées du même membre — mais jamais de cache partagé public pour ces données personnelles (en-tête private, durée courte). L'ordre compte : les actions 1 et 2 corrigent la conception (le vrai problème), l'action 3 soulage la base, l'action 4 polit le résultat ; ajouter un deuxième serveur avant tout cela n'aurait fait que dupliquer un mauvais comportement 301 fois.

# Leçon 7 — Projet : application complète communiquant avec une API

Ce projet assemble tout le cours : livrer, en petite équipe, l'application BiblioIAM complète — un client web et son API — en suivant une démarche professionnelle. La première étape n'est pas de coder, mais de cadrer : utilisateurs (membres, bibliothécaires), périmètre (catalogue, comptes, emprunts et retours ; pas de paiement, pas de réservation dans cette version), et règles métier écrites noir sur blanc (un emprunt pour 14 jours, trois emprunts simultanés au maximum, un livre emprunté ne peut pas l'être deux fois). Ce cadrage devient le contrat de travail de l'équipe et le critère de réussite : le projet est terminé quand chaque règle est démontrée par un test.

La seconde étape conçoit avant de construire : le schéma des données (livres, membres, emprunts et leurs liens), la table complète des routes de l'API avec méthodes, accès et codes de statut, et les maquettes des écrans découpées en composants. Vient ensuite la réalisation en tranches verticales : d'abord le catalogue public (lecture seule), puis les comptes et la connexion par jeton, puis les emprunts protégés, puis l'écran du bibliothécaire. Chaque tranche est testée avant de passer à la suivante, avec un journal d'essais qui note, pour chaque route, la requête envoyée, le code attendu et le code obtenu — y compris les cas d'erreur : 400, 401, 403, 404 et 409 doivent être provoqués volontairement, pas seulement imaginés.

La dernière étape professionnalise le livrable : une documentation courte de l'API (routes, exemples de requêtes et de réponses, codes possibles), un jeu de données de démonstration, et une soutenance où l'équipe fait la démonstration du scénario complet — inscription, connexion, emprunt, retour — puis explique deux choix techniques : pourquoi l'authentification est sans état, et quel réglage de performance a été appliqué. C'est ce type de projet, démontrable et documenté, qui convainc un jury comme un recruteur : il prouve que tu sais non seulement coder, mais concevoir,

sécuriser et expliquer un système entier.

**L'essentiel**
Un projet complet se mène en quatre temps : cadrer (utilisateurs, règles métier), concevoir (données, routes, composants), réaliser par tranches testées (journal d'essais avec tous les codes, erreurs comprises), puis documenter et démontrer. La qualité se prouve par les tests, pas par les promesses.

## Exemple — Les routes du projet BiblioIAM et le scénario complet

```
L'API BiblioIAM du projet (extrait des routes)

Méthode Route Accès OK
GET /v1/livres public 200
GET /v1/livres/{id} public 200
POST /v1/membres public 201
POST /v1/connexion public 200
GET /v1/mes-emprunts membre 200
POST /v1/emprunts membre 201
POST /v1/emprunts/{id}/retour membre 200
POST /v1/livres bibliothécaire 201

Scénario complet, du compte à l'emprunt :
1. POST /v1/membres { nom, email, mot_de_passe }
-> 201 (mot de passe stocké haché, jamais
en clair, et jamais renvoyé au client)
2. POST /v1/connexion { email, mot_de_passe }
-> 200 { jeton } ; mauvais mot de passe -> 401
3. GET /v1/livres -> 200 : Awa repère le livre 7
4. POST /v1/emprunts + jeton { livre_id: 7 }
-> 201, date de retour à 14 jours affichée
5. POST /v1/emprunts/58/retour + jeton -> 200
-> le livre redevient disponible pour tous
```

## À retenir

- Cadrer avant de coder : utilisateurs, périmètre et règles métier écrites servent de critère de réussite.
- Réaliser par tranches verticales testées : catalogue, comptes et jeton, emprunts, écran bibliothécaire.
- Le journal d'essais doit provoquer les erreurs (400, 401, 403, 404, 409), pas seulement les succès.

## Exercice

L'équipe veut ajouter une fonctionnalité au projet : la réservation d'un livre déjà emprunté, qui place le membre dans une file d'attente. Conçois cette extension comme dans la leçon : (1) la ou les routes à ajouter au tableau (méthode, URL, accès, code de réussite) ; (2) deux règles métier à écrire dans le cadrage ; (3) trois lignes du journal d'essais, dont un cas d'erreur, avec le code attendu pour chacune.

**✔ Corrigé**
(1) Routes : POST /v1/livres/{id}/reservations (membre, 201 Created) pour entrer dans la file d'attente d'un livre ; GET /v1/mes-reservations (membre, 200) pour voir ses réservations et sa position ; DELETE /v1/reservations/{id} (membre, 204) pour quitter la file. (2) Règles métier : on ne peut réserver qu'un livre actuellement emprunté (réserver un livre disponible n'a pas de sens : on l'emprunte) ; un membre ne peut pas réserver un livre qu'il détient déjà, ni figurer deux fois dans la même file ; quand le livre revient, le premier de la file est prévenu et le livre lui est gardé 48 heures. (3) Journal d'essais : essai 1 — membre connecté réserve le livre 7 (emprunté) : attendu 201, position 1 dans la file ; essai 2 — réservation du même livre par le même membre : attendu 409 Conflict (« déjà dans la file d'attente ») ; essai 3 — réservation sans jeton : attendu 401 Unauthorized. Bonus cohérent : réserver un livre disponible doit donner 409 aussi (« livre disponible : empruntez-le directement »), ce qui montre que la règle métier a été pensée avant le code.

# Travaux dirigés (TD)

## TD 1 — Concevoir l'API d'une cantine scolaire

Une cantine scolaire veut une API pour son application : les élèves consultent le menu du jour et les plats, un élève connecté réserve un plat (un seul par jour), la cantinière publie le menu et peut voir les réservations du jour, et la direction consulte le nombre de repas servis. Ressources envisagées : plats, menus, reservations, eleves. Travail demandé : 1) dresse la table des routes (méthode, URL, rôle autorisé, code de réussite) pour au moins huit routes couvrant ces besoins ; 2) écris l'échange HTTP complet (requête et réponse) d'une réservation réussie ; 3) donne deux exemples d'erreurs possibles avec leur code et le corps JSON de la réponse.

**✔ Corrigé du TD 1**
Table des routes (une solution) : GET /v1/plats (public, 200) ; GET /v1/menus/jour (public, 200) ; POST /v1/plats (cantinière, 201) ; PUT /v1/menus/jour (cantinière, 200) ; GET /v1/reservations (cantinière ou direction, 200) ; POST /v1/reservations (élève, 201) ; GET /v1/mes-reservations (élève, 200) ; DELETE /v1/reservations/{id} (élève, 204) ; GET /v1/statistiques/repas (direction, 200). Échange de réservation : requête POST /v1/reservations avec l'en-tête Authorization: Bearer <jeton de l'élève> et le corps { "plat_id": 3 } ; réponse 201 Created, en-tête Location: /v1/reservations/87, corps { "id": 87, "plat_id": 3, "eleve_id": 15, "date": "2026-10-12", "statut": "reserve" }. Erreurs : (a) l'élève a déjà réservé aujourd'hui → 409 Conflict, { "erreur": "Une réservation existe déjà pour aujourd'hui" } ; (b) un élève tente de publier un plat (POST /v1/plats) → 403 Forbidden, { "erreur": "Rôle cantinière requis" } — noter la différence avec le 401 : ici l'élève est bien identifié, c'est son droit qui est insuffisant. Bonus accepté : plat inexistant dans la réservation → 404, ou 400 si l'on considère le corps invalide ; les deux se défendent si la convention est documentée et appliquée partout.

## TD 2 — Relire un échange défaillant et le corriger

Voici l'échange réel (anonymisé) d'une petite API de gestion de notes, suivi des remarques d'un auditeur : « POST /api/ajouterNote.php répond 200 "OK" ; GET /api/getNotes?etudiant=5 renvoie aussi le mot de passe de l'étudiant ; un jeton expiré depuis hier est accepté sans message ; DELETE /api/notes/9 sur une note inexistante répond 200 "supprimé" ; les erreurs arrivent tantôt en texte libre, tantôt en page HTML d'erreur du serveur ». Relève au moins cinq défauts distincts, rattache chacun à la règle du cours qu'il viole, et réécris les routes et les comportements corrigés.

**✔ Corrigé du TD 2**
Défaut 1 — verbe et technologie dans l'URL : /ajouterNote.php et /getNotes exposent un verbe et le langage du serveur ; corrigé : POST /v1/notes et GET /v1/etudiants/5/notes (ou GET /v1/notes?etudiant_id=5 pour un filtre). Défaut 2 — mauvais code de création : une création réussie vaut 201 Created avec la ressource créée en réponse, pas 200 "OK" en texte. Défaut 3 — fuite de données sensibles : la réponse ne doit jamais contenir le mot de passe, ni même son empreinte hachée ; on construit la représentation publique de l'étudiant (id, nom, classe) champ par champ, au lieu de renvoyer l'enregistrement brut de la base. Défaut 4 — jeton expiré accepté : la vérification d'un JWT comprend toujours la signature ET la date exp ; un jeton expiré donne 401 et le client doit se reconnecter. Défaut 5 — suppression d'une ressource inexistante annoncée comme réussie : DELETE /v1/notes/9 doit répondre 404 Not Found si la note 9 n'existe pas (un 204 n'est légitime que si la ressource a réellement été supprimée — certaines API acceptent le DELETE idempotent en 204, mais alors il ne faut pas afficher « supprimé » comme un fait). Défaut 6 — erreurs incohérentes : toutes les erreurs suivent le même format JSON { "erreur": "message clair" } avec le code adapté ; une page HTML d'erreur trahit une exception non traitée, qui relève du 500 et doit être journalisée côté serveur. Ces six corrections illustrent le fil du cours : des routes propres, des codes exacts, aucune donnée sensible, un jeton vérifié complètement, un format d'erreur unique.

# Travaux pratiques (TP)

## TP 1 — Construire et éprouver l'API BiblioIAM en local

En binôme, implémentez sur vos postes (langage et framework au choix parmi ceux vus en L2/L3 : PHP, Python ou Node.js) une version locale de l'API BiblioIAM limitée à : GET /v1/livres, GET /v1/livres/{id}, POST /v1/membres, POST /v1/connexion (délivre un JWT signé, valable une heure), POST /v1/emprunts et GET /v1/mes-emprunts (protégées par le jeton). Les mots de passe sont stockés hachés (bcrypt ou fonction équivalente de ton langage). Livrable : le code, plus un journal d'essais au format tableau (route, requête envoyée, code attendu, code obtenu) couvrant au minimum : un succès par route, un 400, un 401 sans jeton, un 401 avec un jeton dont la charge a été modifiée à la main, un 404 et un 409 (double emprunt du même livre).

**✔ Attendus du TP 1**
Attendus et points de contrôle de l'enseignant. Fonctionnel : les six routes répondent depuis un outil de test (navigateur, curl ou Postman) ; l'inscription crée le membre sans jamais stocker le mot de passe en clair — vérification directe dans la base : on doit y lire une empreinte bcrypt/argon2, illisible. Sécurité : la connexion refuse un mauvais mot de passe par un 401 au message identique à celui d'un e-mail inconnu (ne pas révéler lequel des deux est faux) ; le jeton contient sub, iat, exp et le rôle, aucune donnée sensible ; le jeton falsifié (rôle modifié) est rejeté par un 401, ce qui prouve que la signature est réellement vérifiée ; un jeton « expiré » fabriqué avec un exp dans le passé est également rejeté. Métier : le double emprunt du même livre donne 409 ; la date de retour vaut la date d'emprunt + 14 jours. Journal d'essais : au moins onze lignes, toutes au vert (attendu = obtenu), sinon l'écart est expliqué en une phrase. Barème indicatif : routes et codes 40 %, sécurité du jeton et des mots de passe 30 %, règle métier et journal d'essais 30 %. Pièges fréquents à signaler en correction : renvoyer le membre complet (avec l'empreinte) après l'inscription ; oublier de vérifier exp ; confondre 401 et 403 quand un membre tente d'ajouter un livre.

# Exercices corrigés supplémentaires

## Exercice 1

Pour chaque situation, donne le code de statut HTTP correct et justifie en une phrase : (1) un membre crée un emprunt avec succès ; (2) un appel arrive sans jeton sur une route protégée ; (3) un membre connecté tente d'ajouter un livre, action réservée aux bibliothécaires ; (4) on demande le livre n° 999 qui n'existe pas ; (5) le corps JSON de la demande est illisible (accolade manquante) ; (6) un membre tente d'emprunter un livre déjà emprunté par un autre.

**✔ Corrigé**
(1) 201 Created — une ressource (l'emprunt) vient d'être créée ; l'en-tête Location donne son adresse. (2) 401 Unauthorized — l'identité de l'appelant n'est pas prouvée : pas de jeton, pas d'accès. (3) 403 Forbidden — l'identité est prouvée (jeton valide) mais le rôle est insuffisant : c'est la frontière exacte entre authentification et autorisation. (4) 404 Not Found — l'adresse désigne une ressource qui n'existe pas. (5) 400 Bad Request — la demande elle-même est mal formée, le serveur ne peut même pas la comprendre ; ce n'est ni un problème d'identité ni un conflit. (6) 409 Conflict — la demande est correcte et l'appelant identifié, mais elle contredit l'état actuel du système (le livre n'est pas disponible) : c'est le code signature des règles métier « d'état ».

## Exercice 2

Un étudiant a décodé la charge utile d'un jeton reçu par son application et il s'inquiète : { "sub": "7", "nom": "Ibrahim Sissoko", "role": "admin", "mot_de_passe": "soleil123" }. Relève tout ce qui ne va pas dans ce jeton (il y a au moins trois problèmes), en expliquant le risque de chacun et la correction à appliquer côté serveur.

**✔ Corrigé**
Problème 1 — le mot de passe dans la charge utile : un JWT est lisible par quiconque le décode (Base64URL n'est pas un chiffrement) ; ici, le mot de passe en clair de l'utilisateur est donc exposé à tout détenteur du jeton, et même à qui l'intercepte. Correction : ne mettre dans la charge que des affirmations non sensibles (identifiant, nom d'affichage, rôle, dates) — et évidemment ne jamais faire circuler un mot de passe, qui n'existe côté serveur que sous forme d'empreinte hachée. Problème 2 — aucune date d'expiration : sans exp, le jeton est valable indéfiniment s'il est volé ; correction : toujours émettre iat et exp, avec une durée courte (une heure dans notre projet), complétée au besoin par un jeton de rafraîchissement. Problème 3 — le rôle « admin » pour un simple compte d'application : si ce jeton a été émis pour un utilisateur ordinaire promu par erreur, ou falsifié dans un système qui ne vérifierait pas la signature, toutes les routes d'administration sont ouvertes ; correction : le rôle vient de la base au moment de la connexion, jamais d'une donnée fournie par le client, et la signature est vérifiée à chaque requête — c'est elle qui garantit que ce rôle a bien été attribué par le serveur. Point rassurant à expliquer à l'étudiant : pouvoir LIRE la charge est normal et prévu par la norme ; ce qui protège, c'est la signature, pas le secret du contenu.

## Exercice 3

La page « Emprunts du membre » d'une application met 4 secondes à s'afficher. Le journal du serveur montre, pour un seul affichage : 1 requête pour le membre, 1 requête pour ses 25 emprunts, puis 25 requêtes (une par emprunt) pour récupérer le titre de chaque livre. De plus, l'API renvoie les emprunts de toute l'année (environ 400) alors que l'écran n'en affiche que 25. Identifie les deux problèmes de performance, chiffre le nombre de requêtes avant et après correction, et propose la correction côté API pour chacun.

**✔ Corrigé**
Problème 1 — le schéma « N+1 » : après la requête des emprunts, le code fait une requête supplémentaire par emprunt pour trouver le titre du livre. Ici : 1 (membre) + 1 (emprunts) + 400 (titres, puisque l'API renvoie toute l'année) = 402 requêtes pour un écran. Correction : l'API joint le titre du livre à chaque emprunt dès la requête initiale (jointure SQL ou champ dénormalisé dans la représentation) — 2 requêtes au total (membre, emprunts enrichis). Problème 2 — l'absence de pagination : renvoyer 400 emprunts quand l'écran en montre 25 gaspille réseau, mémoire et temps de calcul des deux côtés. Correction : paramètres page et par_page (par exemple page=1&par_page=25) avec le total dans la réponse, pour que le client construise sa navigation. Après correction : 2 requêtes et 25 objets transférés, contre 402 requêtes et 400 objets — l'affichage passe typiquement de plusieurs secondes à une fraction de seconde. Ordre de grandeur à retenir : en performance web, le nombre d'allers-retours coûte souvent plus cher que la taille des réponses ; on optimise d'abord le nombre de requêtes, puis leur volume, puis seulement le reste (cache, matériel).

# QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Dans une API REST, l'adresse correcte pour consulter le livre n° 7 est…

A) GET /recupererLivre/7

B) GET /livres/7

C) GET /livre.php?id=7

2. Un emprunt vient d'être créé avec succès. Le serveur répond…

A) 200 OK

B) 404 Not Found

C) 201 Created

3. À propos de la charge utile d'un JWT, l'affirmation exacte est…

A) Elle est encodée en Base64URL : lisible par tous, mais sa modification invalide la signature

B) Elle est chiffrée : personne ne peut la lire

C) Elle est stockée dans une session côté serveur

4. Un appelant présente un jeton valide de simple membre mais demande une action de bibliothécaire. Le serveur répond…

A) 401 Unauthorized

B) 403 Forbidden

C) 409 Conflict

5. Côté serveur, un mot de passe d'utilisateur doit être conservé…

A) En clair, pour pouvoir le renvoyer en cas d'oubli

B) Chiffré avec une clé rangée à côté de la base

C) Sous forme d'empreinte hachée (bcrypt ou argon2)

6. « API sans état » (stateless) signifie que…

A) L'API n'a pas de base de données

B) Chaque requête contient tout le nécessaire : le serveur ne garde pas de session entre les requêtes

C) Les réponses sont obligatoirement en JSON

7. Dans le découpage d'un serveur en rôles, vérifier le jeton avant d'atteindre les routes protégées est le travail…

A) Du modèle

B) De la base de données

C) D'un intergiciel (middleware)

8. Un client envoie If-None-Match avec l'ETag reçu précédemment et le contenu n'a pas changé. Le serveur répond…

A) 304 Not Modified, sans corps de réponse

B) 500 Internal Server Error

C) 201 Created

9. OAuth 2.0 sert principalement à…

A) Compresser les réponses JSON

B) Déléguer l'authentification à un fournisseur d'identité (connexion via Google, par exemple)

C) Signer les bases de données

10. Dans une application cliente, la donnée qui relève de l'état GLOBAL (partagé) est…

A) Le texte en cours de frappe dans la barre de recherche

B) Le membre connecté et son jeton

C) La couleur d'un bouton pendant son survol

## Corrigés du QCM

1. Réponse B — La ressource est un nom au pluriel (/livres), l'identifiant fait partie de l'adresse, et la méthode GET porte l'action de lecture.

2. Réponse C — 201 Created signale la création d'une ressource ; l'en-tête Location donne l'adresse du nouvel emprunt.

3. Réponse A — Un JWT est signé, pas chiffré : tout le monde peut décoder la charge, mais seul le serveur peut produire une signature valide.

4. Réponse B — 401 = identité non prouvée ; ici elle est prouvée mais le droit est insuffisant : c'est un 403, la frontière entre authentification et autorisation.

5. Réponse C — Le hachage salé et lent (bcrypt, argon2) est irréversible : le serveur compare des empreintes, il ne connaît jamais le mot de passe en clair.

6. Réponse B — Sans état = pas de mémoire de conversation côté serveur ; l'identité voyage avec la requête, dans le jeton. C'est ce qui permet la répartition de charge.

7. Réponse C — Les traitements communs à beaucoup de routes (jeton, journalisation) vivent dans des intergiciels traversés par chaque requête avant le contrôleur.

8. Réponse A — Le 304 dit « ta copie est encore bonne » : aucun contenu n'est retransféré, c'est le cœur du cache HTTP conditionnel.

9. Réponse B — OAuth 2.0 encadre la délégation d'accès : après l'accord de l'utilisateur, le fournisseur d'identité délivre un jeton d'accès à l'application ; OpenID Connect y ajoute la preuve d'identité.

10. Réponse B — Le membre connecté et le jeton servent à tous les écrans et à chaque appel à l'API : ils vivent dans le magasin d'état global. Le reste peut rester local au composant.

# Résumé — les notions importantes

• Architecture moderne : des clients (web, mobile) parlent en HTTP/JSON à une API organisée en couches — API, métier, données — selon un contrat de routes et de formats.

• REST : des ressources nommées par des noms au pluriel, des méthodes HTTP qui portent l'action, aucune session côté serveur, et des filtres ou une pagination en paramètres.

• Les codes de statut racontent le résultat : 200 lecture, 201 création, 400 demande mal formée, 401 identité non prouvée, 403 droit insuffisant, 404 ressource absente, 409 conflit d'état, 500 panne du serveur.

• Un JWT (en-tête . charge . signature, Base64URL) est lisible par tous mais infalsifiable sans la clé secrète ; le serveur vérifie signature et expiration à chaque requête ; les mots de passe ne vivent que hachés (bcrypt/argon2).

• Authentification (« qui es-tu ? ») et autorisation (« as-tu le droit ? ») sont deux contrôles distincts ; OAuth 2.0 délègue l'authentification à un fournisseur d'identité.

• Frameworks : côté serveur, routage, contrôleurs minces, modèles et intergiciels ; côté client, composants réutilisables et état séparé en local (composant) et global (membre, jeton).

• Performances : mesurer, supprimer les requêtes N+1, paginer, puis cacher (Cache-Control, ETag, 304) ; l'absence d'état rend possible la montée en charge horizontale derrière un répartiteur.

# Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

## Leçon 2 — Les API REST — Ce que tout dev junior doit absolument comprendre

Chaîne : Kaly Labs · Langue : Français · Niveau : Intermédiaire

Une synthèse claire des fondamentaux REST : méthodes HTTP, URLs sans verbes, codes de statut et pattern CRUD, illustrés par une vraie API en Node.js. Parfait pour consolider la leçon 2 avant de concevoir tes propres routes.

https://www.youtube.com/watch?v=rVEW9Kbj9ig

## Leçon 2 — L'ATELIER DU CODE #3 : TOUT COMPRENDRE À LA REST API (avec @llcenlive )

Chaîne : Titom NCL · Langue : Français · Niveau : Débutant à intermédiaire

Un cours en mode atelier qui reprend les principes et contraintes de REST, les verbes HTTP et la structure des endpoints, puis passe à la pratique. L'exercice d'inventaire proposé à la fin prolonge bien le TD de ce cours.

https://www.youtube.com/watch?v=9iElJFKdltg

## Leçon 3 — Spring Security — Comment ca marche ? JWT, BCrypt, Filter Chain | Java Spring Boot

Chaîne : lioncode · Langue : Français · Niveau : Intermédiaire

En quelques minutes, le flux complet d'authentification d'une API Java : génération et validation du jeton JWT, hachage BCrypt des mots de passe et autorisation par rôles. Idéal pour voir la leçon 3 appliquée dans un framework réel.

https://www.youtube.com/watch?v=C3ByHBBXCi4

## Leçon 4 — Spring Framework — C'est quoi ? Cours Complet en Français

Chaîne : lioncode · Langue : Français · Niveau : Intermédiaire
Un cours complet sur le framework serveur le plus utilisé en Java : organisation du code, contrôleurs REST, couche données et sécurité. Il montre concrètement les conventions et le découpage en rôles décrits dans la leçon 4.

https://www.youtube.com/watch?v=tCPy11Nf-8w

### Leçon 7 — Node JS Tutorial Français pour Débutant - Cours complet 8h [2022]

Chaîne : Codeur Senior · Langue : Français · Niveau : Débutant à intermédiaire
Un projet fil rouge complet en Node.js : construction d'une API REST, liaison à une base SQL, authentification JWT puis déploiement. C'est le compagnon pratique idéal du projet de la leçon 7, à suivre avec ton éditeur ouvert.

https://www.youtube.com/watch?v=NRxzvpdduvQ

## Sources et références

• RFC 9110 — « HTTP Semantics », IETF : la définition officielle des méthodes et des codes de statut utilisés dans ce cours.

• RFC 7519 — « JSON Web Token (JWT) », IETF : la norme qui définit la structure en-tête . charge utile . signature.

• RFC 6749 — « The OAuth 2.0 Authorization Framework », IETF : le cadre de délégation d'authentification présenté en leçon 3.

• Documentation MDN Web Docs (developer.mozilla.org/fr) — les pages « HTTP » et « Codes de statut de réponse HTTP », en accès libre.

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !